État de l'art des attaques contre les ML-NIDS

1. Périmètre et classification

Un *ML-NIDS* est un système de détection d'intrusions réseau qui utilise l'apprentissage automatique. Il ne regarde pas les paquets un par un : il les résume d'abord en *caractéristiques*, par exemple la durée d'une connexion, le nombre de paquets, la taille des échanges ou les drapeaux TCP. Un modèle (SVM, réseau de neurones, autoencodeur, réseau sur graphes…) reçoit ces caractéristiques et décide si le trafic est normal ou malveillant.

Une attaque *adversariale* ne vise pas directement le réseau : elle vise ce détecteur. Il faut donc bien séparer deux choses : l'activité malveillante elle-même (un scan, un déni de service, une force brute…) et l'attaque contre le modèle, dont le but est de cacher cette activité ou de rendre le détecteur inutilisable.

On distingue deux familles selon le moment où l'attaquant agit :

— **Évasion :** le modèle est déjà entraîné. L'attaquant modifie son trafic au moment de l'attaque pour qu'il soit classé comme normal.
— **Empoisonnement :** l'attaquant agit *avant*, sur les données d'entraînement (exemples ou étiquettes). Le modèle apprend alors de mauvaises règles. Une **porte dérobée** (*backdoor*) est un cas particulier d'empoisonnement : le modèle fonctionne normalement, sauf quand il voit un motif secret, appelé *déclencheur*, choisi par l'attaquant.

Il faut aussi préciser ce que l'attaquant sait du modèle : **boîte blanche** (il connaît le modèle et ses paramètres), **boîte grise** (il en connaît une partie) ou **boîte noire** (il ne voit que les réponses). En boîte noire, il peut interroger le détecteur, ou entraîner son propre modèle « substitut » et espérer que l'attaque fonctionne aussi sur le vrai [1, 5, 6]. Enfin, il faut dire ce qu'il contrôle : seulement son propre trafic, des machines compromises, les données d'entraînement, ou l'accès aux prédictions.

2. Évasion : tromper un modèle déjà entraîné

Modifier directement les caractéristiques. TIKI-TAKA teste cinq attaques en boîte noire (NES, Boundary, Pointwise, HopSkipJump et OPT) contre des MLP, CNN et C-LSTM [6]. Le principe est simple : l'attaquant envoie beaucoup de requêtes au détecteur et ajuste petit à petit le vecteur de caractéristiques jusqu'à ce qu'il soit classé bénin. Une variante n'autorise à modifier que les variables liées au temps. Mao et al. ajoutent des règles pour que le vecteur modifié reste cohérent (méthode SPTS) et cherchent des perturbations qui marchent sur plusieurs modèles à la fois (méthode DGM) [5]. Ils repassent ensuite au niveau des paquets. Limite commune : un vecteur qui trompe le modèle ne correspond pas forcément à du vrai trafic réalisable, ni à une attaque qui fonctionne encore.

Changer la forme du trafic. Ici, on touche directement aux paquets. TrafficManipulator retarde des paquets et en ajoute d'autres pour que les statistiques observées ressemblent à du trafic normal, notamment contre KitNET, un ensemble d'autoencodeurs [1]. TANTRA apprend, avec un LSTM, le rythme des paquets dans le trafic bénin, puis applique ce rythme au trafic malveillant sans toucher à son contenu [2]. Attention : le LSTM sert à *générer* l'attaque, ce n'est pas lui le détecteur ; les cibles sont KitNET, un autoencodeur et Isolation Forest. Le prix à payer est du retard, ce qui gêne les attaques qui ont besoin de débit, comme un déni de service.

Se servir de l'explicabilité. Okada et al. utilisent KernelSHAP pour savoir quelles caractéristiques comptent le plus dans la décision d'un MLP en boîte noire, puis modifient le trafic pour changer justement celles-là [3]. Par exemple, pour une force brute FTP, ils jouent sur le drapeau PSH ou répartissent les essais de mot de passe dans plusieurs sessions TCP. Ils testent aussi des attaques XSS entre machines virtuelles et vérifient que l'attaque marche toujours après modification. C'est l'étude la plus proche d'une démonstration réelle, car elle relie la tromperie du modèle et le fonctionnement de l'attaque réseau.

Modifier le graphe des communications. Certains détecteurs (E-GraphSAGE, LineGraphSAGE) voient le réseau comme un graphe : les machines sont des nœuds et les communications des arêtes. Venturi et al. modifient ce graphe [4]. Dans la variante C2x_B, un hôte compromis ajoute des communications normales autour du flux malveillant, sans retirer ce dernier : son « voisinage » change et le modèle peut se tromper. Ces expériences restent toutefois faites sur des flux et des graphes, pas sur un réseau actif.

3. Empoisonnement

L'idée est de « mal éduquer » le détecteur. Comme il apprend à partir d'exemples, l'attaquant glisse de faux exemples ou de fausses étiquettes dans ses données d'entraînement. Il existe deux objectifs. Soit on veut **dégrader** le détecteur en général : il fait plus d'erreurs et devient peu fiable. Soit on veut installer une **porte dérobée** : le détecteur reste bon en apparence, mais laisse passer le trafic qui porte le déclencheur. Dans tous les cas, l'attaquant doit pouvoir toucher aux données d'entraînement, ou au processus d'entraînement lui-même. Envoyer du trafic à un modèle figé ne suffit pas : il faut qu'il y ait un entraînement ou un réentraînement après l'injection.

Exemples contradictoires (Venkatesan et al.). La cible est un SVM qui sépare le trafic bénin des scans Nmap [7]. L'attaquant ajoute des exemples étiquetés « scan » mais placés juste à côté d'exemples bénins que le modèle reconnaît très bien. Au réentraînement, le modèle reçoit des informations contradictoires : sa frontière de décision se déplace et il fait plus d'erreurs, surtout des faux positifs (du trafic normal pris pour une attaque). Le but n'est donc pas de faire passer un flux précis, mais de rendre le détecteur pénible à utiliser. Les auteurs gardent des valeurs de caractéristiques cohérentes, mais ne rejouent pas l'attaque sur un vrai réseau.

Changer les étiquettes (Das et al.). Ici, l'attaquant ne crée pas de nouveaux exemples : il change les étiquettes d'une partie des données existantes [8]. Pour choisir par quelle étiquette remplacer chaque classe, il calcule un « profil moyen » par classe et compare ces profils par similarité cosinus. Avec *Max-CSLM*, une classe est remplacée par la classe qui lui ressemble le plus : le changement est plus discret. Avec *Min-CSLM*, elle est remplacée par la classe la plus différente : c'est plus visible, mais fait plus de dégâts. Un paramètre fixe la part d'étiquettes modifiées (10 % dans leur scénario principal). Les tests portent sur un réseau SDN avec un ou trois contrôleurs, le jeu de données UNR-IDD et trois modèles (Random Forest, SVM et MLP). Avec un seul contrôleur, la performance chute d'environ 50 % pour le Random Forest (Min-CSLM) et de 60 % pour le SVM (Max-CSLM). L'hypothèse forte est que l'attaquant a compromis un contrôleur SDN et peut donc modifier les étiquettes : contrôler du trafic ne suffit pas.

Empoisonner une source externe (Alhussien et Aleroud). Beaucoup d'organisations ont peu de trafic étiqueté. Elles entraînent donc leur détecteur avec des jeux de données externes, plus quelques exemples internes (apprentissage *few-shot*). L'attaquant n'a pas besoin d'entrer chez la victime : il lui suffit d'empoisonner une des sources externes [9]. Il choisit des exemples d'attaque et les rapproche légèrement de flux bénins jugés très représentatifs, pour que le modèle confonde les deux. Seuls 5 % des flux de la source sont empoisonnés. Sur CIC-IDS2018 et UNSW-NB15, avec une régression logistique, le taux de détection des attaques passe de 79 % à 47 %, sans hausse notable des faux positifs. Quand l'organisation ajoute plus d'exemples internes étiquetés, l'effet de l'attaque diminue nettement : c'est une piste de défense simple.

Porte dérobée sans changer les étiquettes (PCAP-Backdoor). PCAP-Backdoor ajoute des paquets spécialement construits, le déclencheur, dans une partie du trafic bénin qui sert à l'entraînement [10]. Les étiquettes restent vraies (bénin reste bénin) : on parle d'empoisonnement à *étiquettes propres*, ce qui le rend difficile à repérer. Le modèle associe alors le déclencheur à la classe « bénin ». Plus tard, l'attaquant ajoute le même déclencheur à son trafic malveillant, qui est alors classé bénin. Sur du trafic sans déclencheur, le modèle reste bon : une évaluation classique ne voit rien. L'étude travaille directement sur des fichiers PCAP et teste plusieurs modèles et jeux de données IoT et industriels.

Porte dérobée par un entraînement sous-traité (Ma et al.). Dans ce scénario, la victime confie l'entraînement de son détecteur IoT à un tiers malveillant, puis vérifie seulement le modèle rendu sur ses propres données de test [11]. Le tiers prend 0,1 % des exemples d'entraînement, met quelques caractéristiques à leur valeur maximale (c'est le déclencheur) et les étiquette « bénin ». Sur CICIoT2023 et six modèles (AdaBoost, GBDT, XGBoost, MLP, CNN et RNN), le trafic malveillant avec déclencheur passe inaperçu dans plus de 97 % des cas dès quatre caractéristiques modifiées, alors que les performances sur trafic normal baissent de moins de 3 % : le modèle passe donc les tests de la victime. Les auteurs proposent aussi une défense : repérer le lien anormalement fort entre déclencheur et classe bénigne, puis faire « désapprendre » la porte dérobée, ce qui ramène le taux de réussite sous 0,5 %. Limite : le déclencheur est défini sur les caractéristiques, pas sur des paquets réels.

4. Enseignements, défenses et critères d'évaluation

Ces travaux montrent qu'on ne peut pas parler de la robustesse d'un modèle seul. Ce qui compte, c'est le **couple modèle–extracteur** (le modèle et la manière dont les caractéristiques sont calculées), ainsi que ce que l'attaquant peut faire. Par exemple, les modèles sur graphes peuvent être trompés en ajoutant des communications, et les détecteurs basés sur le temps sont sensibles aux changements de rythme. Les résultats obtenus sur Kitsune, CICIDS, TON_IoT, CTU-13 ou CICIoT2023 dépendent du protocole de chaque étude : on ne peut pas comparer directement des taux de réussite venant d'articles différents.

Côté défense, on trouve l'entraînement adversarial (entraîner le modèle avec des exemples d'attaque), les ensembles de modèles, la détection des requêtes répétées [1, 6], le nettoyage des données d'entraînement quand plusieurs classifieurs ne sont pas d'accord [7], l'ajout de données internes fiables [9] et le désapprentissage des portes dérobées [11]. PCAP-Backdoor montre cependant que certaines défenses, comme le regroupement des activations du réseau, ne repèrent pas tous les déclencheurs [10]. Chaque défense protège contre des scénarios précis ; aucune ne garantit une robustesse générale.

Pour le démonstrateur, chaque expérience doit comparer un état « avant attaque » et un état « après attaque », en mesurant au moins : le **rappel sur le trafic malveillant** (la part d'attaques détectées), les **faux positifs**, et le **coût de l'attaque** (retard ajouté, volume de trafic, nombre de requêtes ou part des données empoisonnées). Pour un taux d'évasion, il faut préciser sur quoi il est calculé, par exemple seulement sur les flux qui étaient détectés au départ. Pour une porte dérobée, il faut mesurer séparément la réussite avec déclencheur et la qualité du modèle

sans déclencheur. Enfin, après toute modification des paquets, il faut recalculer les caractéristiques et vérifier que l'activité réseau fonctionne encore. Pour l'évasion, TrafficManipulator/KitNET, TANTRA et les scénarios d'Okada sont de bonnes pistes complémentaires. Pour l'empoisonnement, le SVM de Venkatesan et le changement d'étiquettes de Das illustrent la dégradation, tandis que PCAP-Backdoor et l'attaque de Ma et al. illustrent les portes dérobées.

Références du corpus

[1] D. Han et al. (2021). *Evaluating and Improving Adversarial Robustness of Machine Learning-Based Network Intrusion Detectors*. IEEE JSAC. doi:10.1109/JSAC.2021.3087242.

[2] Y. Sharon et al. (2022). *TANTRA : Timing-Based Adversarial Network Traffic Reshaping Attack*. IEEE TIFS. doi:10.1109/TIFS.2022.3201377.

[3] S. Okada et al. (2025). *Xai-driven black-box adversarial attacks on network intrusion detectors*. IJIS. doi:10.1007/s10207-025-01016-0.

[4] A. Venturi, D. Stabili et M. Marchetti (2024). *Problem space structural adversarial attacks for Network Intrusion Detection Systems based on Graph Neural Networks*. arXiv:2403.11830v2.

[5] Z. Mao et al. (2025). *Exploring transferable adversarial attacks for Deep Learning-based Network Intrusion Detection*. JNCA. doi:10.1016/j.jnca.2025.104255.

[6] C. Zhang, X. Costa-Pérez et P. Patras (2022). *Adversarial Attacks Against Deep Learning-Based Network Intrusion Detection Systems and Defense Mechanisms*. IEEE/ACM ToN. doi:10.1109/TNET.2021.3137084.

[7] S. Venkatesan et al. (2021). *Poisoning Attacks and Data Sanitization Mitigations for Machine Learning Models in Network Intrusion Detection Systems*. MILCOM. doi:10.1109/MILCOM52596.2021.9652916.

[8] T. Das, R. M. Shukla et S. Sengupta (2025). *Poisoning the Well : Adversarial Poisoning on ML-Based Software-Defined Network Intrusion Detection Systems*. IEEE TNSE. doi:10.1109/TNSE.2024.3492032.

[9] N. Alhussien et A. Aleroud (2023). *A Novel Poisoning Attack on Few-Shot based Network Intrusion Detection*. IEEE/IFIP NOMS. doi:10.1109/NOMS56928.2023.10154453.

[10] A. Koyatan Chathoth et al. (2026). *PCAP-Backdoor : Backdoor Generator in Network Traffic for Intrusion Detection Systems*. ACM TCPS. doi:10.1145/3805033.

[11] B. Ma et al. (2025). *Backdoor Attack and Defense Methods for AI-Based IoT Intrusion Detection System*. IET Information Security. doi:10.1049/ise2/6664900.